



## FDA 文件提取智能體：從正則到自癒 AI 的演進

# 從一個資料工程的踩坑經驗開始

## 個人與背景資訊

- 林志忠 Data Service Manager & GenAI Strategist
- 世博科技顧問
- 智慧財產 × 法規情報
- 政府公開資料 → 可訂閱的商業情報
- 今天：一個系統失效 → 一套方法論

你的公司，現在是否也面臨這三個問題？

**PDF 格式一改**  
提取程式就壞掉

**三層式抽取 + 幾何**  
座標定位，格式不再是痛點

**需要抽取新欄位**  
要從頭重寫一套規則

**自癒 Agent 代理**  
生成並驗證提取程式

**AI 直接給答案**  
不知道答案能不能信

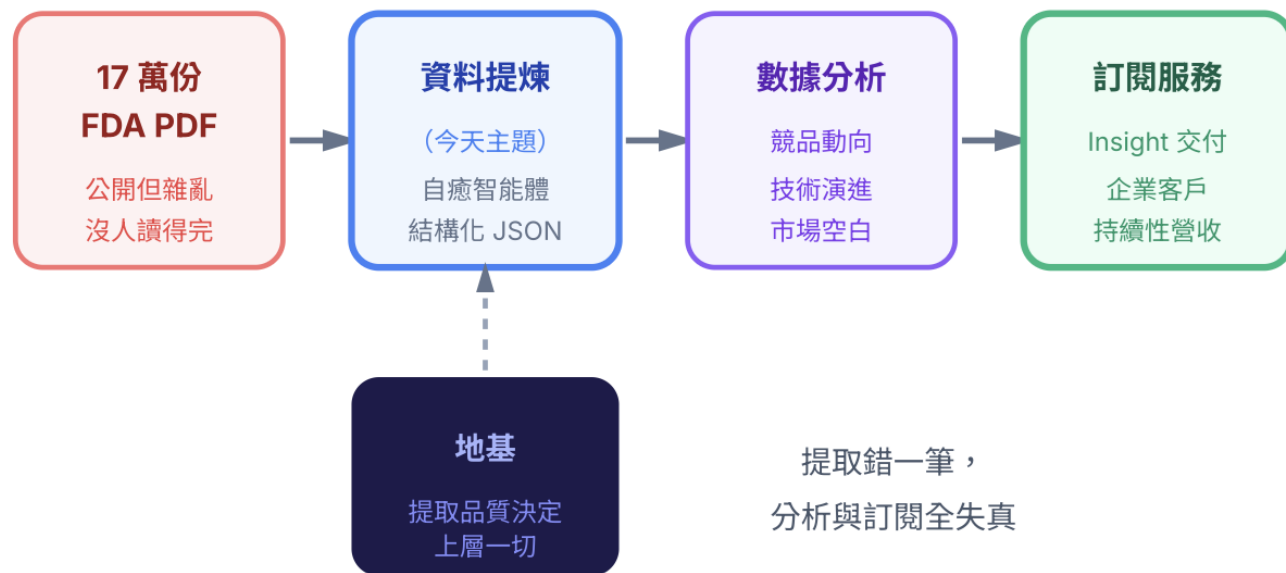
**讓 AI 寫出可跑、可測**  
可版控、自行驗證的程式

# 我們為什麼要處理 17 萬份 FDA 文件？

一個 Alert，我們重新審視整個架構

- FDA 510(k)：美國醫材進市場的許可證
- 30 年 × 17 萬份 公開，全是 PDF
- 提取失效 = 情報失真 = 訂閱服務信譽歸零
- 工程師的挑戰：格式一改，程式要怎麼不崩？

## 從原始 PDF 到訂閱營收的價值鏈



## 補充背景：FDA 510(k) 文件是什麼？

- FDA 510(k) = 醫療器材進入美國市場的「准入申請書」，全球公開，30 年歷史，數十萬份 PDF。
- 每份文件的三個關鍵欄位：① 適應症 (IFU，長文非結構化) ② 器材描述 ③ 前代器材 K 碼 (格式 K123456)。
- 商業價值：人工審閱 1 份 = 120 分鐘；AI 提取 = 5 秒。效率差距 1,440 倍，且覆蓋法規、競品、知識庫三大應用。

### FDA 510(k) Document — 三個我們要提取的欄位

#### ① 適應症 Indications for Use

非結構化長文本，決定器材合規範圍，最難提取

#### ② 器材描述 Device Description

硬體規格與工作原理，半結構化

#### ③ 前代器材 K-Number (格式 K123456)

建立器材血緣圖譜，格式固定但 OCR 常出錯

實際產出：<https://iximedtech.com/>

## 某一天：一次系統失效的經驗

一個空格，三天工程工時

- 提取率：95% → 23%
- 原因：K-number 前面多了一個空格
- 代價：三天工程工時
- 反思：規則永遠補不完





# 雙路架構：三層Fallback + 自癒 Agent

IFU 走管線，K-number 走自癒 Agent，兩條路

- IFU 適應症：Tier 1 → Tier 2 → Tier 3 Fallback
- K-number：自癒 Agent → AI 自動產出程式碼
- 兩條路完全獨立，服務不同目的
- 換文件類型 × 換目標欄位，骨架不動

兩條提取路徑，服務不同目的

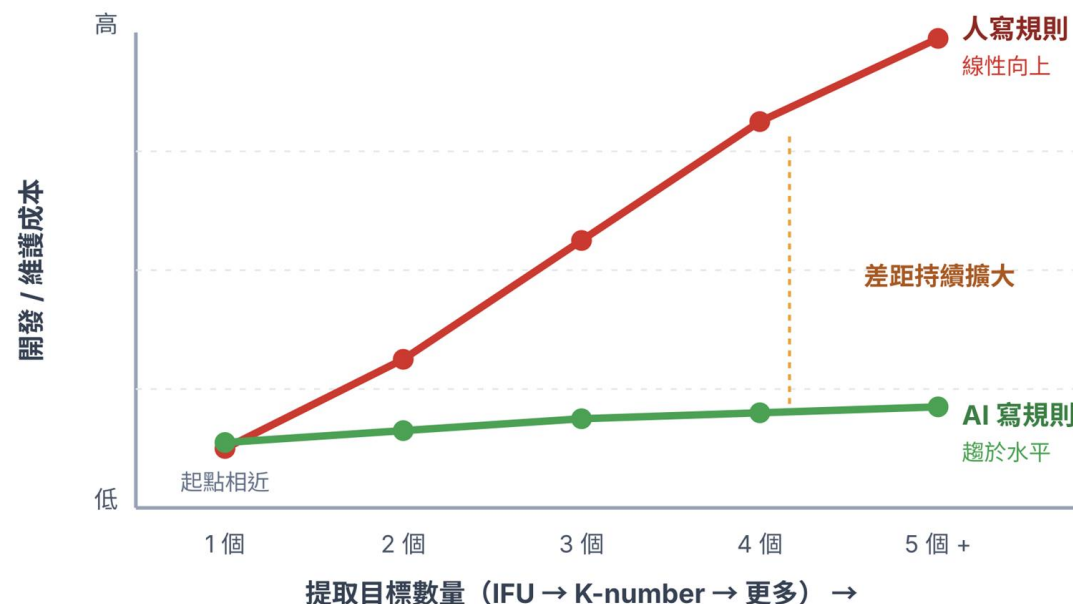


## 轉捩點：「人寫規則」到「AI 寫規則」

從「補了三十條規則」到「讓 AI 自己寫程式」

- 第一階段：IFU 適應症 → run\_rule\_processor.py
- 遭遇：數十條定錨規則，每條都是一步一腳印踩出來
- 新需求：K-number → 另一套完全不同的邏輯
- 第二階段：auto\_healing\_extractor.py → AI 自己寫程式

### 從原始 PDF 到訂閱營收的價值鏈

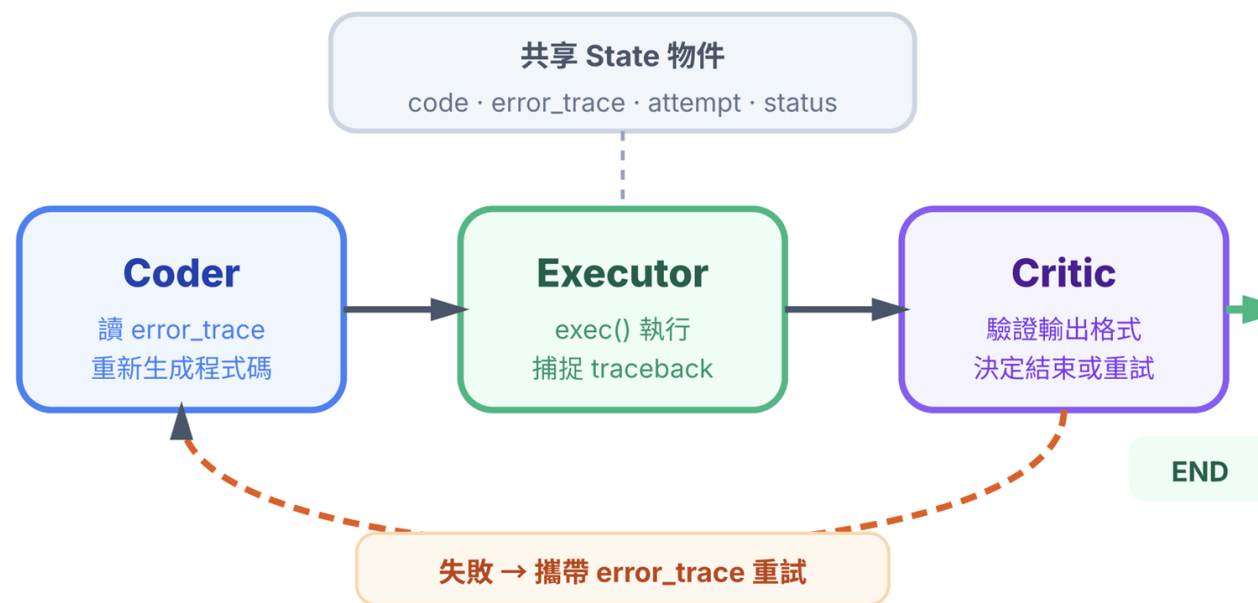


## 自癒架構：讓 AI 自行除錯

Coder → Executor → Critic，一個不斷迭代的迴圈

- Coder：讀 error\_trace，重新生成程式碼
- Executor：exec() 執行，捕捉 traceback
- Critic：驗證輸出格式，決定結束或重試
- 上限：5 次仍未收斂 → 進入人工審核

## 自癒Workflow：AI自己寫程式、執行、驗證



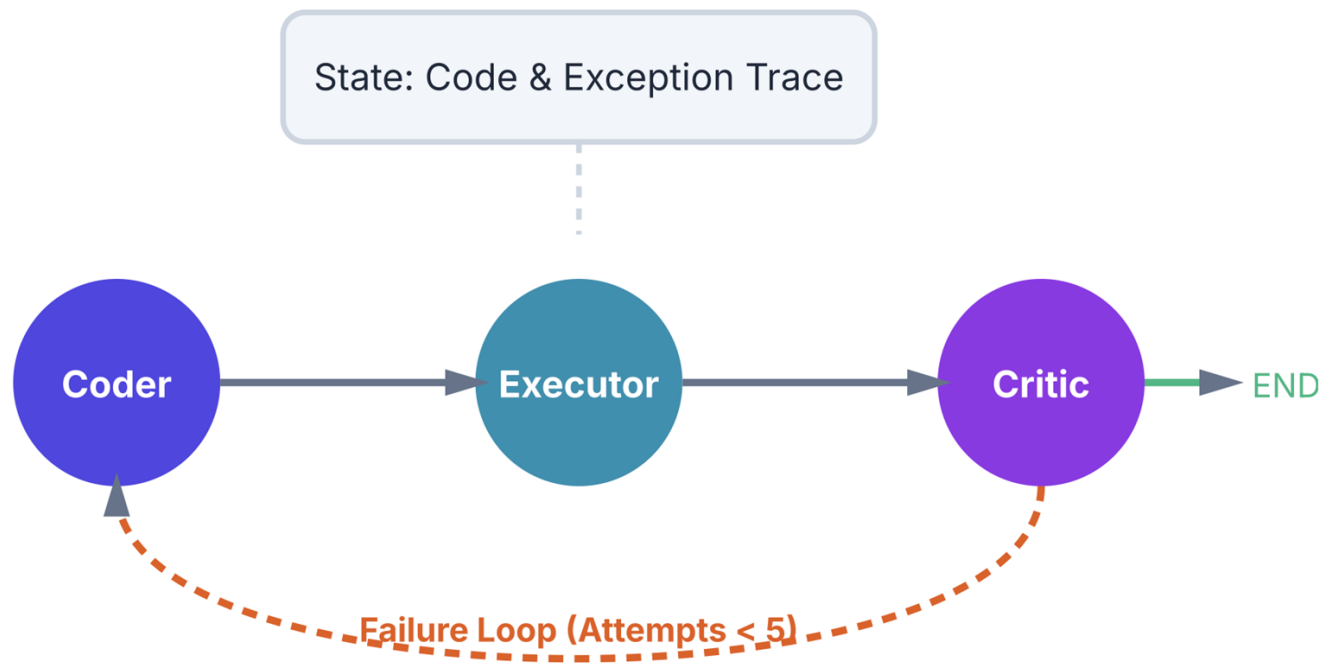


# 為什麼需要 LangGraph ?

為什麼 while loop 不夠，State 物件才是關鍵

- while loop：每次重試，上次報錯消失
- LangGraph State：error\_trace 共享
- 條件路由：Critic 通過 → END / 失敗 → 攜帶報錯回 Coder
- 自癒迴圈（方法論）× LangGraph（技術手段）

迭代進化：每次失敗，系統變得更聰明一點



# 自癒迴圈最終寫出了什麼？

Agent 自己選了座標定位，不是工程師教它的

- 這段程式不是人工手寫的
- Agent自己選了：座標定位 > 純 Regex
- 定錨位置：向下 150px + 左右 80px
- IFU → 規則式 / K-number → 幾何座標

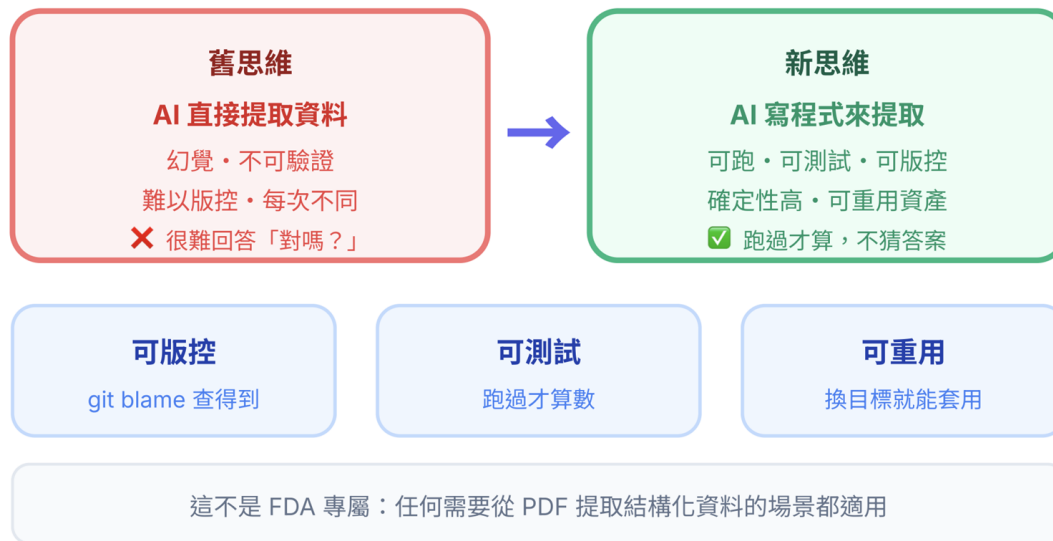
```
# final_universal_extractor.py
# Agent 自癒後產出的核心邏輯（非人工手寫）
for anchor in anchors:          # anchor = 頁面上「predicate」的位置
    search_box = {
        'x0': anchor['x0'] - 80,
        'x1': anchor['x1'] + 80,
        'top': anchor['bottom'],
        'bottom': anchor['bottom'] + 150    # 往下搜尋 150px
    }
    # 在此座標範圍內找 K\d{6}
```

# 讓 AI 寫程式來提取資料

一個觀念的轉換，解鎖了 4 萬份文件的處理能力

- 4萬+ 份 FDA 文件，已完成結構化提取
- 舊思維：AI 直接回答 → 幻覺、不可驗證
- 新思維：AI 寫程式 → 可跑、可測試、可重用
- 腳本是持久的工程資產，不是一次性的 AI 回答

目前成效：4 萬+ 份文件已完成結構化提取



# 能解決什麼問題、什麼適合用

你的 Data/PDF 有多亂，這套架構就有多值錢

- 混亂 PDF → 結構化 例：JSON
- 關鍵差異：失敗時能自己修好
- 自建的三個條件：量大 × 格式亂 × 抽錯代價高
- 少量 + 格式固定 → 直接用現成工具



適用性自我檢測（三項都符合才適合自建）

文件量大且持續進來？

符合

格式雜亂又常改版？

符合

抽錯代價高 / 有合規要求？

符合

# 你的業務裡，哪裡有相同的痛？

誰在半夜改規則打補丁？

- 法務合約：改版就斷，工程師打補丁不是在審合約
- 財務報表：供應商換格式，欄位錯位需手動複核
- 企業 RAG：整份 PDF 進庫，AI 把雜訊一起引用

## 「痛點」及「解方」

| 業務場景                               | 你的痛點                           | 解方                                      |
|------------------------------------|--------------------------------|-----------------------------------------|
| <b>法務合約審查</b><br>Regex 每季改版<br>就斷掉 | 補規則補丁不是在審合約，<br>而是在維護程式        | 語意視窗定位條款段落<br>+ 自癒 Agent 生成<br>新格式的提取程式 |
| <b>財務報表處理</b><br>供應商換格式<br>欄位就錯位   | 財務人員手動複核<br>每一筆，<br>高錯誤率低效率    | 幾何座標定位金額欄<br>格式換了座標<br>通常不變             |
| <b>企業 RAG</b><br>知識庫建構<br>雜訊進、雜訊出  | AI 把頁首制式廢話<br>一起引用，<br>答案離題不可信 | Boilerplate 清洗<br>+ 段落邊界切割<br>讓進庫的都有意義  |



## 回去之後，如何開始？

不需要打造整套系統，從換掉一支腳本開始

- Step 1：找出最常壞掉的那支 PDF 提取腳本
- Step 2：包成 Coder → Executor → Critic 三節點
- Step 3：讓 Agent 自癒一次，觀察它改了什麼
- 起點是換掉一支腳本，不是重寫整個系統

### Step 1 找出最常壞掉的那支 PDF 提取腳本

問自己：哪支 Regex 最常因格式改版失效，讓工程師半夜起來修？



### Step 2 包成 Coder → Executor → Critic 三節點

用 LangGraph 建自癒迴圈，讓 AI 寫程式、跑程式、自己改程式



### Step 3 讓 Agent 自癒一次，觀察它改了什麼

檢視產出的腳本：AI 選了什麼提取策略？修了哪個 bug？

# 成本與限制

做到了什麼、什麼做不到、何時才值得自建

- 限制①：修好 A 壞 B，5 次上限後交人工審核
- 限制②：Critic 誤判通過，污染下游資料
- 成本：地端  $\neq$  \$0，GPU 折舊 + 維運人力不能忽略
- 量大  $\times$  格式亂  $\times$  有隱私合規要求  $\rightarrow$  **才值得自建**

## 過來人的建議～

### ⚠ 系統限制（誠實說）

#### 自癒震盪

修好 A 壞 B，5 次耗盡  
 $\rightarrow$  人工審核佇列

#### 靜默失敗風險

Critic 誤判通過  
 $\rightarrow$  悄悄污染下游資料

#### 待補強

os 模組白名單尚未收緊

### 💰 真實成本結構

#### 隱形成本（別漏算）

GPU 折舊 + 電費  
模型維運 + 沙盒基礎設施

#### 損益平衡點

量大  $\times$  格式亂  
 $\times$  有隱私合規要求

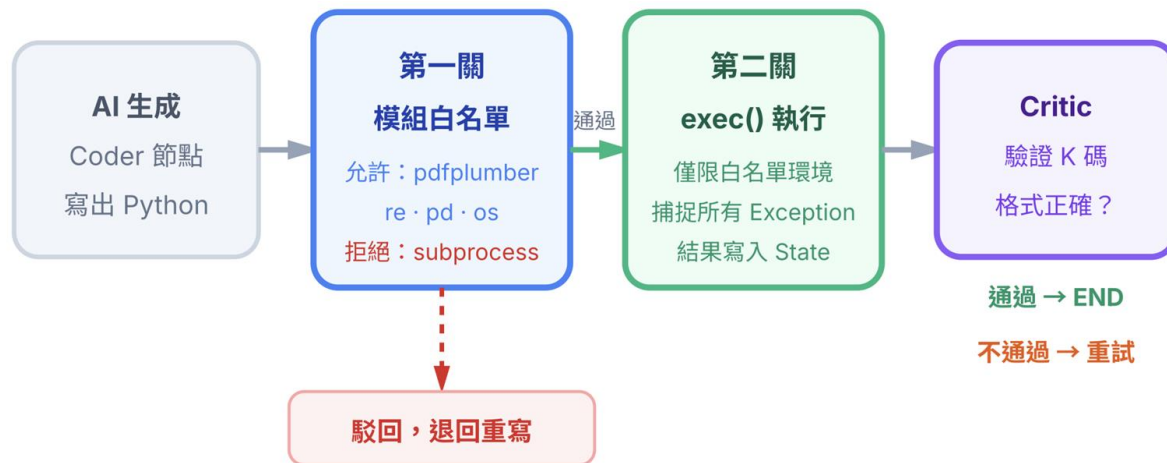
#### 不適合自建

一次性小批量  $\rightarrow$  雲端 API

# AI 產生的程式碼如何控管風險

目前還欠缺的：做到了什麼、還沒做到什麼

- 風險：AI 寫的程式透過 `exec()` 執行
- 做法：只傳入 `pdfplumber · re · pd · os` 四個模組
- `os` 仍在白名單 → 後續待加入靜態掃描
- Critic：輸出格式驗證，不合格就退回重寫



待補強：os 模組目前仍在白名單，計畫加入靜態程式碼分析  
(在 `exec()` 前掃描語法，攔截危險操作)

# 謝謝您的聆聽！

請掃描下方連結

- 核心：讓 AI 寫程式，不是讓 AI 給答案
- IFU 三層 Fallback / K-number 自癒生成程式



[medium.com/@kahnlin](https://medium.com/@kahnlin)



[www.linkedin.com/in/kahnlin](https://www.linkedin.com/in/kahnlin)

今天帶走一句話

**讓 AI 寫程式，  
不是讓 AI 給答案。**

## IFU 適應症

三層 Fallback 管線

規則 → 地端 LLM → 雲端 LLM

## K-number

自癒 Agent 生成程式

`auto_healing_extractor.py`